e.laverdeb@uniandes.edu.co_parcial1

March 10, 2026

1 Parcial 1: Caso de estudio - Potabilidad del agua

Autor: Estefanía Laverde Becerra

Fecha: 2026-03-08

1.1 1. Exploración de los datos para identificar problemas de calidad y otras características.

En primero lugar, definimos las librerías a utilizar.

[130]:
```
# lectura del dataset
import pandas as pd

# graficos
import seaborn as sns
import matplotlib.pyplot as plt

# reporte de datos
from ydata_profiling import ProfileReport

# preprocesamiento
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OrdinalEncoder, OneHotEncoder
from sklearn.model_selection import train_test_split

# modelos
from sklearn.svm import SVC
from sklearn.tree import plot_tree
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix,␣
↪accuracy_score, recall_score, precision_score, f1_score

# ignorar warnings
import warnings
```

```python
warnings.filterwarnings('ignore')
```

A continuación, cargamos el conjunto de datos y realizamos una exploración inicial de las 5 primeras filas para entender su estructura y contenido.

[131]:
```python
# lectura de datos
data_potability = pd.read_csv('dataset/Datos_examen11.csv', sep=';')

# obtener tamaño del dataset
print(f'Tamaño del dataset: {data_potability.shape}')

# visualizamos las primeras filas del dataset
data_potability.head()
```

```
Tamaño del dataset: (10500, 19)
```

[131]:
```
   Sample_ID  Temperature    pH  Salinity  Turbidity  Dissolved_Oxygen  \
0   WQ00000        15.86  8.34     30.91      29.96              4.50
1   WQ00001        26.13  8.01     39.32      41.14             12.90
2   WQ00002        26.70  6.51     36.79      24.72             12.94
3   WQ00003        21.27  8.70     35.21      30.38              4.70
4   WQ00004        19.37  8.65     30.69      10.06             13.80

   Nitrate  Chlorophyll_a  nutrient_index  pH_deviation  \
0    11.73          70.10        822.2730          1.34
1     1.30          19.64         25.5320          1.01
2     0.52          10.00          5.2000          0.49
3    12.78          75.55        965.5290          1.70
4     3.13          24.02         75.1826          1.65

   Rainfall_last_7_days_mm  Distance_to_pollution_source_km  Water_depth_m  \
0                   44.945                           18.995        146.540
1                  114.086                           16.979         38.533
2                      NaN                            9.220         70.635
3                   71.839                           30.560        133.330
4                   18.722                           24.093         97.454

   Current_velocity_m_s  Phosphate_concentration_mg_L  BOD_mg_L  \
0                 0.961                        0.1530     8.549
1                 0.694                        0.0519     5.198
2                 1.447                        0.0675     2.357
3                 0.336                        0.0944     7.498
4                 0.886                        0.1058     4.477

   Distance_to_coast_km Seagrass_or_algae_cover  is_safe
0                74.181                  Medium        0
1                88.122                    High        1
2                46.372                     Low        1
```

```
3                 28.989                    Medium        0
4                 31.953                    Medium        1
```

Con la primera exploración, podemos observar que el conjunto de datos tiene un total de 10500 registros y 19 columnas. De estas columnas, tenemos 16 caractersíticas numéricas sobre atributos que describen la calidad de muestras de agua, una columna categórica (`Seagrass_or_algae_cover`), una columna de identificación (`Sample_ID`) y una columna objetivo (`is_safe`) que indica si el agua es potable (1) o no potable (0).

Para obtener una visión más detallada sobre todas las columnas y las relaciones entre ellas, utilizamos el reporte de perfilado de datos con ydata_profiling. Este reporte nos permite identificar posibles problemas de calidad de datos, como valores faltantes, distribuciones de las variables, correlaciones entre características y la distribución de la variable objetivo.

[132]:
```
profile = ProfileReport(data_potability, title="Potability Dataset Report")
profile.to_notebook_iframe()
```

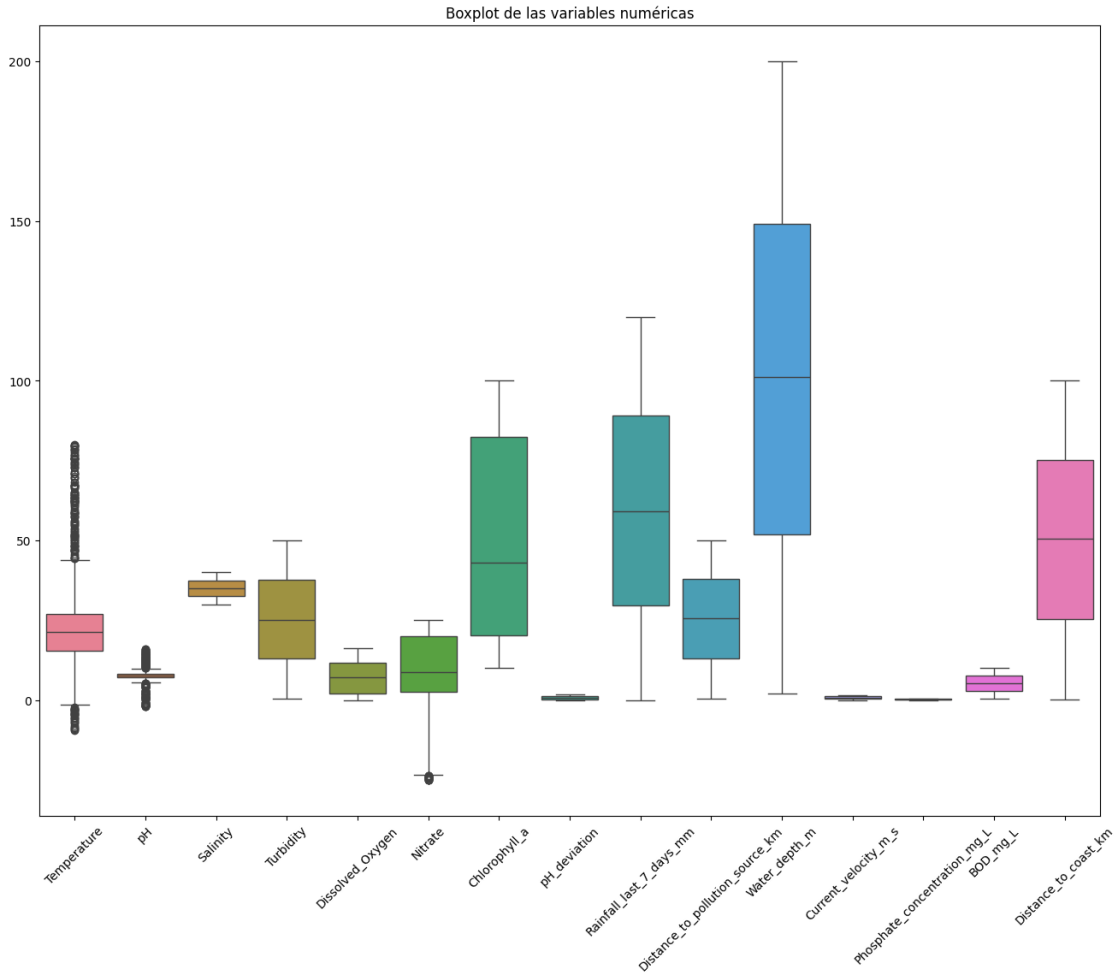
```
Summarize dataset:   0%|          | 0/5 [00:00<?, ?it/s]

100%|      | 19/19 [00:00<00:00, 205.94it/s]

Generate report structure:   0%|          | 0/1 [00:00<?, ?it/s]

Render HTML:   0%|          | 0/1 [00:00<?, ?it/s]

<IPython.core.display.HTML object>
```

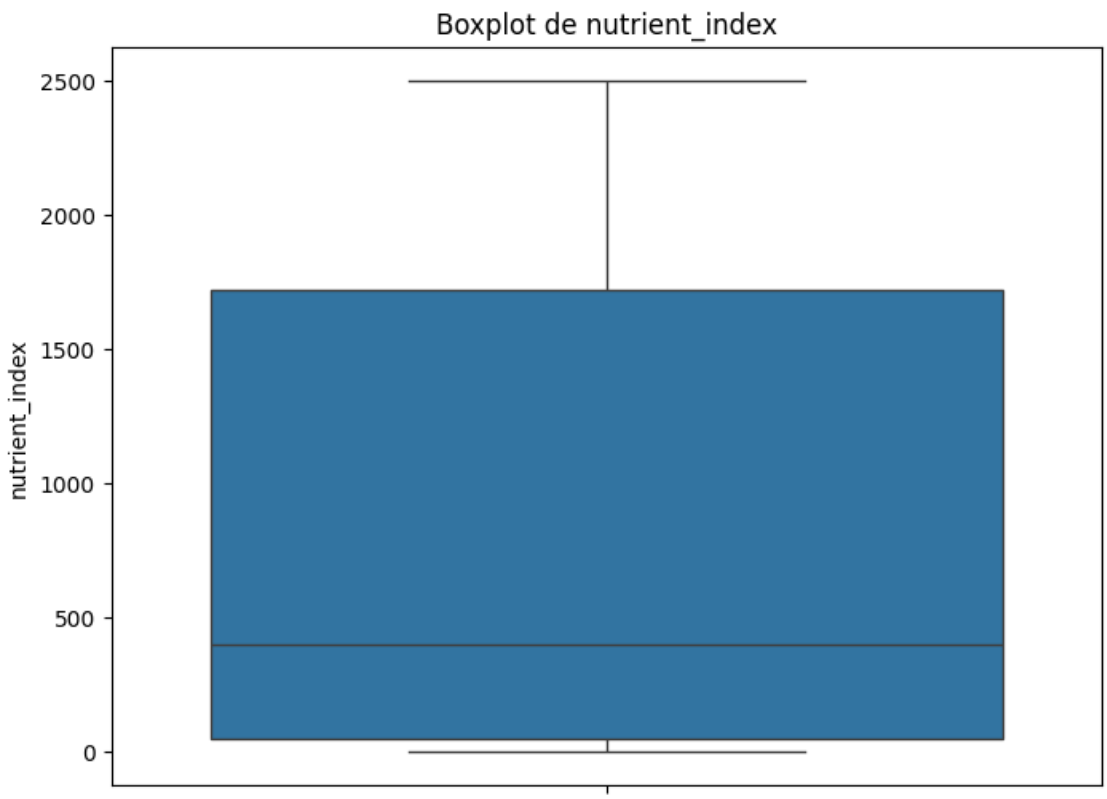
Haciendo uso del reporte de ydata_profiling, tenemos las siguientes observaciones: - El dataset tiene 2557 valores faltantes, lo que representa el 1.33% del total de los datos. Las columnas con más valores faltantes son `Temperature`, `Nitrate`, `Rainfall_last_7_days_mm` y `Water_depth_m`. - Las columnas `nutrient_index`, `Nitrate` y `Dissolved_Oxygen` presentan valores 0, lo que podría indicar la presencia de datos faltantes o errores en la recolección de datos. - La variable objetivo tiene una distribución equilibrada, con 5252 muestras de agua potable (1) y 5248 muestras de agua no potable (0). - Hay duplicados en el dataset, exactamente 439 filas, lo que representa el 4.2% del total de los datos. - Con respecto a las correlaciones, se observan correlaciones altas entre: - `Clorophyll_a` con `Dissolved_Oxygen`, `Nitrate`, `is_safe` y `nutrient_index`. - `Dissolved_Oxygen` con `Nitrate`, `is_safe` y `nutrient_index`. - `Nitrate` con `is_safe` y `nutrient_index`. - `is_safe` con `nutrient_index`. - `pH` con `pH_deviation`.

A continuación, creamos un boxplot sobre las variables para identificar posibles outliers, especialmente, para reconocer si los valores 0 en las columnas `nutrient_index`, `Nitrate` y `Dissolved_Oxygen` corresponden a outliers o a datos faltantes, o si son parte de la distribución normal de los datos.

[133]:
```
# boxplot para identificar outliers - nutrient index se saca por escala
plt.figure(figsize=(16, 12))
sns.boxplot(data=data_potability.drop(columns=['Sample_ID',␣
 ↪'Seagrass_or_algae_cover', 'is_safe', 'nutrient_index']))
plt.xticks(rotation=45)
plt.title('Boxplot de las variables numéricas')
plt.show()
```

```python
# boxplot para nutrient index
plt.figure(figsize=(8, 6))
sns.boxplot(data=data_potability['nutrient_index'])
plt.title('Boxplot de nutrient_index')
plt.show()
```





Con estas gráficas observamos que los valores 0 en las columnas `nutrient_index`, `Nitrate` y `Dissolved_Oxygen` no corresponden a outliers, sino que parecen ser parte de la distribución normal de los datos al encontrarse dentro del rango de cada variable. Sin embargo, la variable `nutrient_index` aparenta tener un valor casi único para cada muestra. Esto se puede comprobar contando las apariciones de cada valor.

[134]:
```
data_potability.value_counts('nutrient_index')
```

[134]:
```
nutrient_index
0.0000      303
28.1000       5
21.0000       5
18.8000       4
34.3000       4
            …
99.3616       1
99.3408       1
99.1752       1
99.1485       1
503.9100      1
Name: count, Length: 9543, dtype: int64
```

Con esto comprobamos que, en general, cada muestra tiene un valor único de dicha variable, por

lo que no aporta información relevante al modelo para distinguir entre muestras de agua potable y no potable, y por lo tanto, se podría considerar eliminar esta variable.

1.2 2. Propuesta de limpieza y preparación de los datos, justificando las decisiones tomadas con base en los resultados obtenidos en el paso anterior y de acuerdo con los modelos que se van a construir

Con base en las observaciones obtenidas en el paso anterior y los problemas de calidad encontrados, se proponen dos pasos de limpieza de datos: general y específico para cada modelo. A continuación, se describen los pasos generales de limpieza utilizados para todos los modelos.

1. **Eliminar las filas duplicadas**, ya que estas pueden sesgar los resultados de los modelos y no aportan información adicional.
2. **Imputar los valores faltantes utilizando la media**, pues los valores faltantes se presentan únicamente variables numéricas.
3. Se van a **mantener los valores 0** en las columnas `nutrient_index`, `Nitrate` y `Dissolved_Oxygen`, asumiendo que estos valores son reales con base en las distribuciones observadas. Además, se considera que estas tres variables son importantes para la clasificación, por lo que eliminarlas podría afectar el rendimiento de los modelos.
4. **Eliminar la columna `Sample_ID`**, ya que es un identificador único para cada muestra y no aporta información relevante para la clasificación.
5. **Eliminar la columna `nutrient_index`**, ya que, en general, cada muestra tiene un valor único de dicha variable, por lo que no aporta distinguibilidad entre clases.

Teniendo en cuenta los modelos a construir, se proponen también preparaciones adicionales de los datos:

1. Para la **regresión logística**, dado a que la multicolinealidad puede afectar no solo el rendimiento del modelo al no poder determinar el efecto individual de cada variable, sino también la interpretación de los coeficientes, se propone **eliminar variables altamente correlacionadas**, específicamente, las variables `Dissolved_Oxygen`, `nutrient_index`, `Nitrate` y `pH_deviation`, manteniendo únicamente con `Clorophyll_a` y `pH`, pues estas no están correlacionadas entre sí, y `Clorophyll_a` está correlacionada con la variable objetivo, lo que puede ayudar en el rendimiento del modelo. Además, `Clorophyll_a` es un indicador indirecto de la salud del ecosistema acuático, ya que niveles altos de clorofila sugieren eutrofización (crecimiento incontrolado de algas), que a su vez reduce el oxígeno disuelto y degrada la calidad del agua. Adicionalmente, como la regresión logística es sensible a la escala de las variables, pues puede afectar la convergencia del modelo al tratarse de un proceso de optimización, se propone **estandarizar las variables** numéricas utilizando la técnica de estandarización (z-score normalization). Finalmente, para la variable categórica `Seagrass_or_algae_cover`, se observa una relación de orden ('Low', 'Medium', 'High'). Por un lado, la codificación ordinal (0, 1, 2) permitiría al modelo aprovechar esta jerarquía, aunque asume que el salto de importancia entre cada nivel es uniforme y constante. Por otro lado, la técnica de One-Hot Encoding trata cada categoría como una dimensión independiente, evitando imponer una métrica de distancia predefinida y permitiendo que el modelo detecte si un nivel específico tiene un impacto desproporcionado respecto a los demás. Dado que no hay certeza de cuál representación del espacio de características es óptima, **se incluirán ambas técnicas (OrdinalEncoder y One-HotEncoder) dentro de la búsqueda de hiperparámetros**.

2. Para el modelo **SVM**, la presencia de multlicolinealidad no afecta directamente el proceso

de optimización para encontrar el hiperplano que maximiza el margen de separación, por lo que se opta por **conservar las variables correlacionadas**, priorizando la completitud de la información disponible sobre la simplificación forzada del modelo. Esta decisión también está respaldada por el hecho de que SVM es un algoritmo diseñado para trabajar eficazmente en espacios de alta dimensionalidad. En cuanto a la escala, es crítico **aplicar estandarización** de las variables numéricas, ya que SVM maximiza el margen de separación basándose en distancias euclidianas; sin el escalado, las variables con magnitudes mayores dominarían la función de decisión, invalidando el aporte de las variables con rangos menores. Finalmente, para la variable `Seagrass_or_algae_cover`, se determinará por medio de la búsqueda de hiperparámetros si se utiliza codificación ordinal o One-Hot Encoding, siguiendo la **misma lógica que para la regresión logística**.

3. Para el modelo de **árboles de decisión** se optará por **usar el conjunto de datos con todas las variables**, sin eliminar las variables altamente correlacionadas, puesto a que los árboles de decisión son modelos que realizan particiones basadas en la importancia de las variables, por lo que no se ven afectados por la multicolinealidad. Además, los árboles de decisión **no requieren que las variables estén en la misma escala**, por lo que no es necesario aplicar estandarización. Finalmente, para la variable `Seagrass_or_algae_cover`, se optará por usar una **codificación ordinal**, puesto a que puede separar una del resto de categorías en un solo nodo, mientras que con One-Hot Encoding se necesitarían múltimples ramas para entender una sola variable, lo que lo haría menos eficiente.

Antes de continuar con los procesos de limpieza específicos, se eliminan duplicados y la columna de identificación, y luego se separan las variables en explicativas y objetivo, y se realiza la división del dataset en conjuntos de entrenamiento y prueba, con un 80% para entrenamiento y un 20% para prueba, utilizando una semilla aleatoria de 77 para garantizar la reproducibilidad de los resultados.

Este paso se hace previo a la eliminación de valores faltantes, estandarización y eliminación de columnas específicas para cada modelo de forma que el conjunto de prueba se mantenga completamente independiente del proceso de limpieza y preparación de los datos, evitando así cualquier tipo de fuga de información que pueda sesgar la evaluación del modelo. Además, al fijar previamente el conjunto de prueba, se asegura que la comparación entre los modelos, independiente de la preparación de datos, sea comparable.

[135]:
```
# eliminar filas duplicadas, columna de identificación y columna␣
↪`nutrient_index`
data_potability = data_potability.drop_duplicates()
data_potability = data_potability.drop(columns=['Sample_ID', 'nutrient_index'])

# conjuntos X e y
X = data_potability.drop(columns=['is_safe'])
y = data_potability['is_safe']

# división en conjuntos de entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
↪random_state=77)
```

Ahora definimos los pipelines de limpieza y preparación de datos para cada modelo.

[136]:
```python
# tomar columnas categóricas y numéricas
categorical_features = X_train.select_dtypes(include=['object']).columns
numerical_features = X_train.select_dtypes(include=['number']).columns

# -- REGRESION LOGISTICA --
# definir las columnas a eliminar y las que se mantienen
cols_to_drop = ["Dissolved_Oxygen", "nutrient_index", "Nitrate", "pH_deviation"]
numerical_features_filtered = [col for col in numerical_features if col not in␣
↪cols_to_drop]

# pipeline para variables numéricas
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler()),
])

# pipeline para variables categóricas
cat_pipeline = Pipeline([
    ('encoder', OneHotEncoder(handle_unknown='ignore')) # este se cambiará por␣
↪OrdinalEncoder en la búsqueda de hiperparámetros del modelo de regresión␣
↪logística
])
# preprocesador para regresión logística
preprocessor_logreg = ColumnTransformer([
    ('num', num_pipeline, numerical_features_filtered),
    ('cat', cat_pipeline, categorical_features)
],remainder='drop') # se deja en drop para que no pasen columnas no deseadas

# -- SVM --
# pipeline para svm de columnas numéricas
num_pipeline_svm = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler()),
])

# preprocesador para SVM - utilizo el mismo pipeline de variables categóricas␣
↪que para logreg
preprocessor_svm = ColumnTransformer([
    ('num', num_pipeline_svm, numerical_features), # aqui uso todas las columnas
    ('cat', cat_pipeline, categorical_features)
],remainder='drop') # se deja en drop para que no pasen columnas no deseadas

# -- DECISION TREE --
# pipeline para variables numéricas
num_pipeline_tree = Pipeline([
    ('imputer', SimpleImputer(strategy='mean'))
])
```

```python
# pipeline para variables categóricas
cat_pipeline_tree = Pipeline([
    ('encoder', OrdinalEncoder(categories=[['Low', 'Medium', 'High']])) # se␣
 ↪ordenan las categorías para que el modelo de árbol de decisión tenga en␣
 ↪cuenta la relación de orden entre ellas
])

# preprocesador
preprocessor_tree = ColumnTransformer([
    ('num', num_pipeline_tree, numerical_features),
    ('cat', cat_pipeline_tree, categorical_features)
],remainder='drop')
```

Comprobamos el procesamiento de limpieza en cada caso.

[137]:
```python
# comprobación en regresión logística y SVM
X_train_logreg_processed = preprocessor_logreg.fit_transform(X_train)
X_train_logreg_processed_df = pd.DataFrame(X_train_logreg_processed,␣
 ↪columns=preprocessor_logreg.get_feature_names_out())
X_train_logreg_processed_df.head()
```

[137]:
```
   num__Temperature   num__pH  num__Salinity  num__Turbidity  \
0          0.407194 -0.562596      -0.598653        1.513615
1         -0.713099  0.830587      -1.106305       -0.450426
2          0.319948 -0.798207       0.672204       -1.716234
3          0.584216  0.390096      -0.118628        0.497877
4         -3.620846 -0.101616      -1.272069       -0.012801

   num__Chlorophyll_a  num__Rainfall_last_7_days_mm  \
0           -1.277579                 -3.847313e-01
1           -0.799250                  1.577725e+00
2           -0.690108                  2.119807e-16
3            1.358922                  1.441683e+00
4           -0.813486                  4.085448e-01

   num__Distance_to_pollution_source_km  num__Water_depth_m  \
0                             -0.078439            0.806822
1                              0.121743           -1.745360
2                             -0.394205           -0.747620
3                             -1.101911            0.569641
4                              0.933056            0.719209

   num__Current_velocity_m_s  num__Phosphate_concentration_mg_L  \
0                   0.358597                          -0.257741
1                  -0.089043                           1.196724
2                   1.096160                           0.546098
```

```
3                   0.551106                          -1.051085
4                  -1.183791                          -0.337495

   num__BOD_mg_L  num__Distance_to_coast_km  \
0      -1.024997                   0.322017
1      -1.188272                  -1.283847
2       0.892290                  -0.125123
3      -0.684203                  -0.114965
4      -0.357654                  -1.158746

   cat__Seagrass_or_algae_cover_High  cat__Seagrass_or_algae_cover_Low  \
0                                0.0                                1.0
1                                0.0                                1.0
2                                0.0                                1.0
3                                0.0                                1.0
4                                0.0                                0.0

   cat__Seagrass_or_algae_cover_Medium
0                                  0.0
1                                  0.0
2                                  0.0
3                                  0.0
4                                  1.0
```

[138]:
```
X_train_logreg_processed_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8051 entries, 0 to 8050
Data columns (total 15 columns):
 #   Column                                Non-Null Count  Dtype
---  ------                                --------------  -----
 0   num__Temperature                      8051 non-null   float64
 1   num__pH                               8051 non-null   float64
 2   num__Salinity                         8051 non-null   float64
 3   num__Turbidity                        8051 non-null   float64
 4   num__Chlorophyll_a                    8051 non-null   float64
 5   num__Rainfall_last_7_days_mm          8051 non-null   float64
 6   num__Distance_to_pollution_source_km  8051 non-null   float64
 7   num__Water_depth_m                    8051 non-null   float64
 8   num__Current_velocity_m_s             8051 non-null   float64
 9   num__Phosphate_concentration_mg_L     8051 non-null   float64
 10  num__BOD_mg_L                         8051 non-null   float64
 11  num__Distance_to_coast_km             8051 non-null   float64
 12  cat__Seagrass_or_algae_cover_High     8051 non-null   float64
 13  cat__Seagrass_or_algae_cover_Low      8051 non-null   float64
 14  cat__Seagrass_or_algae_cover_Medium   8051 non-null   float64
dtypes: float64(15)
memory usage: 943.6 KB
```

[139]:
```python
# comprobación en SVM
X_train_svm_processed = preprocessor_svm.fit_transform(X_train)
X_train_svm_processed_df = pd.DataFrame(X_train_svm_processed,␣
↪columns=preprocessor_svm.get_feature_names_out())
X_train_svm_processed_df.head()
```

[139]:
```
   num__Temperature   num__pH  num__Salinity  num__Turbidity  \
0          0.407194 -0.562596      -0.598653        1.513615
1         -0.713099  0.830587      -1.106305       -0.450426
2          0.319948 -0.798207       0.672204       -1.716234
3          0.584216  0.390096      -0.118628        0.497877
4         -3.620846 -0.101616      -1.272069       -0.012801

   num__Dissolved_Oxygen  num__Nitrate  num__Chlorophyll_a  num__pH_deviation  \
0               1.319808     -0.819357           -1.277579          -1.231047
1               0.680581     -0.563484           -0.799250           1.308246
2               0.792446     -0.648043           -0.690108          -1.175033
3              -0.725718      1.344034            1.358922           0.505381
4               0.340992     -0.567877           -0.813486          -0.390840

   num__Rainfall_last_7_days_mm  num__Distance_to_pollution_source_km  \
0                 -3.847313e-01                              -0.078439
1                  1.577725e+00                               0.121743
2                  2.119807e-16                              -0.394205
3                  1.441683e+00                              -1.101911
4                  4.085448e-01                               0.933056

   num__Water_depth_m  num__Current_velocity_m_s  \
0            0.806822                   0.358597
1           -1.745360                  -0.089043
2           -0.747620                   1.096160
3            0.569641                   0.551106
4            0.719209                  -1.183791

   num__Phosphate_concentration_mg_L  num__BOD_mg_L  \
0                          -0.257741      -1.024997
1                           1.196724      -1.188272
2                           0.546098       0.892290
3                          -1.051085      -0.684203
4                          -0.337495      -0.357654

   num__Distance_to_coast_km  cat__Seagrass_or_algae_cover_High  \
0                   0.322017                                0.0
1                  -1.283847                                0.0
2                  -0.125123                                0.0
3                  -0.114965                                0.0
4                  -1.158746                                0.0
```

```
   cat__Seagrass_or_algae_cover_Low  cat__Seagrass_or_algae_cover_Medium
0                               1.0                                  0.0
1                               1.0                                  0.0
2                               1.0                                  0.0
3                               1.0                                  0.0
4                               0.0                                  1.0
```

[140]:
```
X_train_svm_processed_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8051 entries, 0 to 8050
Data columns (total 18 columns):
 #   Column                                Non-Null Count  Dtype
---  ------                                --------------  -----
 0   num__Temperature                      8051 non-null   float64
 1   num__pH                               8051 non-null   float64
 2   num__Salinity                         8051 non-null   float64
 3   num__Turbidity                        8051 non-null   float64
 4   num__Dissolved_Oxygen                 8051 non-null   float64
 5   num__Nitrate                          8051 non-null   float64
 6   num__Chlorophyll_a                    8051 non-null   float64
 7   num__pH_deviation                     8051 non-null   float64
 8   num__Rainfall_last_7_days_mm          8051 non-null   float64
 9   num__Distance_to_pollution_source_km  8051 non-null   float64
 10  num__Water_depth_m                    8051 non-null   float64
 11  num__Current_velocity_m_s             8051 non-null   float64
 12  num__Phosphate_concentration_mg_L     8051 non-null   float64
 13  num__BOD_mg_L                         8051 non-null   float64
 14  num__Distance_to_coast_km             8051 non-null   float64
 15  cat__Seagrass_or_algae_cover_High     8051 non-null   float64
 16  cat__Seagrass_or_algae_cover_Low      8051 non-null   float64
 17  cat__Seagrass_or_algae_cover_Medium   8051 non-null   float64
dtypes: float64(18)
memory usage: 1.1 MB
```

[141]:
```
# comprobación en DecisionTree
X_train_tree_processed = preprocessor_tree.fit_transform(X_train)
X_train_tree_processed_df = pd.DataFrame(X_train_tree_processed,␣
 ↪columns=preprocessor_tree.get_feature_names_out())
X_train_tree_processed_df.head()
```

[141]:
```
   num__Temperature  num__pH  num__Salinity  num__Turbidity  \
0         24.790000     7.10          33.25           46.64
1         15.930000     8.46          31.78           18.68
2         24.100000     6.87          36.93            0.66
3         26.190000     8.03          34.64           32.18
4         -7.066344     7.55          31.30           24.91
```

```
   num__Dissolved_Oxygen  num__Nitrate  num__Chlorophyll_a  num__pH_deviation  \
0                  13.67          3.34               10.46               0.10
1                  10.47          5.67               25.58               1.46
2                  11.03          4.90               29.03               0.13
3                   3.43         23.04               93.80               1.03
4                   8.77          5.63               25.13               0.55

   num__Rainfall_last_7_days_mm  num__Distance_to_pollution_source_km  \
0                     46.261000                                24.426
1                    112.041000                                27.301
2                     59.156894                                19.891
3                    107.481000                                 9.727
4                     72.851000                                38.953

   num__Water_depth_m  num__Current_velocity_m_s  \
0             145.594                      0.907
1               4.750                      0.714
2              59.811                      1.225
3             132.505                      0.990
4             140.759                      0.242

   num__Phosphate_concentration_mg_L  num__BOD_mg_L  \
0                             0.2153          2.449
1                             0.4232          2.002
2                             0.3302          7.698
3                             0.1019          3.382
4                             0.2039          4.276

   num__Distance_to_coast_km  cat__Seagrass_or_algae_cover
0                     59.645                           0.0
1                     13.639                           0.0
2                     46.835                           0.0
3                     47.126                           0.0
4                     17.223                           1.0
```

[142]:
```
X_train_tree_processed_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8051 entries, 0 to 8050
Data columns (total 16 columns):
 #   Column                                Non-Null Count  Dtype
---  ------                                --------------  -----
 0   num__Temperature                      8051 non-null   float64
 1   num__pH                               8051 non-null   float64
 2   num__Salinity                         8051 non-null   float64
 3   num__Turbidity                        8051 non-null   float64
 4   num__Dissolved_Oxygen                 8051 non-null   float64
```

```
 5   num__Nitrate                          8051 non-null   float64
 6   num__Chlorophyll_a                    8051 non-null   float64
 7   num__pH_deviation                     8051 non-null   float64
 8   num__Rainfall_last_7_days_mm          8051 non-null   float64
 9   num__Distance_to_pollution_source_km  8051 non-null   float64
 10  num__Water_depth_m                    8051 non-null   float64
 11  num__Current_velocity_m_s             8051 non-null   float64
 12  num__Phosphate_concentration_mg_L     8051 non-null   float64
 13  num__BOD_mg_L                         8051 non-null   float64
 14  num__Distance_to_coast_km             8051 non-null   float64
 15  cat__Seagrass_or_algae_cover          8051 non-null   float64
dtypes: float64(16)
memory usage: 1006.5 KB
```

1.3 3. Desarrollar un modelo de regresión logística incorporando una búsqueda de hiperparámetros con validación cruzada, explorando el tipo de regularización y el solver utilizado. Este proceso debe estructurarse mediante un pipeline, de manera que el preprocesamiento y el modelo queden integrados en un flujo de trabajo reproducible y ordenado.

En primer lugar, se define la grilla de hiperparámetros a explorar para la regresión logística, que incluye: - Tipo de regularización: L1, L2, ElasticNet y None. - Solver: lbfgs, liblinear, newton-cg, newton-cholesky, sag, saga. - C (penalización): 0.001, 0.01, 0.1, 1, 10 y 100. - Codificación de la variable categórica "Seagrass_or_algae_cover": One-Hot Encoding y Ordinal Encoding.

Como nota adicional, hay que tener en cuenta que no todas las combinaciones de solver y tipo de regularización son compatibles, pero el GridSearchCV se encarga internamente de manejar estas incompatibilidades.

[ ]:
```python
# creamos el pipeline para regresión logística
pipeline_logreg = Pipeline([
    ('preprocessor', preprocessor_logreg), # se usa el preprocessor para variar␣
 ↪la codificación de la variable categórica
    ('classifier', LogisticRegression(max_iter=10000))
])

# creamos el grid de parametros para penalty y C
param_grid = {
    'classifier__penalty': ['l1', 'l2', 'elasticnet', 'none'],
    'classifier__solver': ['lbfgs', 'liblinear', 'newton-cg',␣
 ↪'newton-cholesky', 'sag', 'saga'],
    'classifier__C': [0.001, 0.01, 0.1, 1, 10, 100],
    'preprocessor__cat__encoder': [
        OneHotEncoder(handle_unknown='ignore'),
        OrdinalEncoder(categories=[['Low', 'Medium', 'High']]) # los tipos de␣
 ↪categoría se ordenan para que tenga en cuenta la relación de orden
    ]
}
```

```
# hacer la búsqueda con gridseach
grid_search = GridSearchCV(
    estimator=pipeline_logreg,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy', # usar accuracy como métrica de evaluación y 5 folds
    n_jobs=-1, # usar todos los núcleos disponibles para acelerar la búsqueda
)

grid_search.fit(X_train, y_train)
```

Obtenemos el mejor modelo y los mejores hiperparámetros con la validación cruzada utilizando GridSearchCV, con 5 folds y usando accuracy como métrica de evaluación.

[144]:
```
# obtener el mejor modelo y parámetros
best_params_logreg = grid_search.best_params_
best_model_logreg = grid_search.best_estimator_

print("Best Parameters:")
for param_name, param_value in best_params_logreg.items():
    print(f"{param_name}: {param_value}")
```

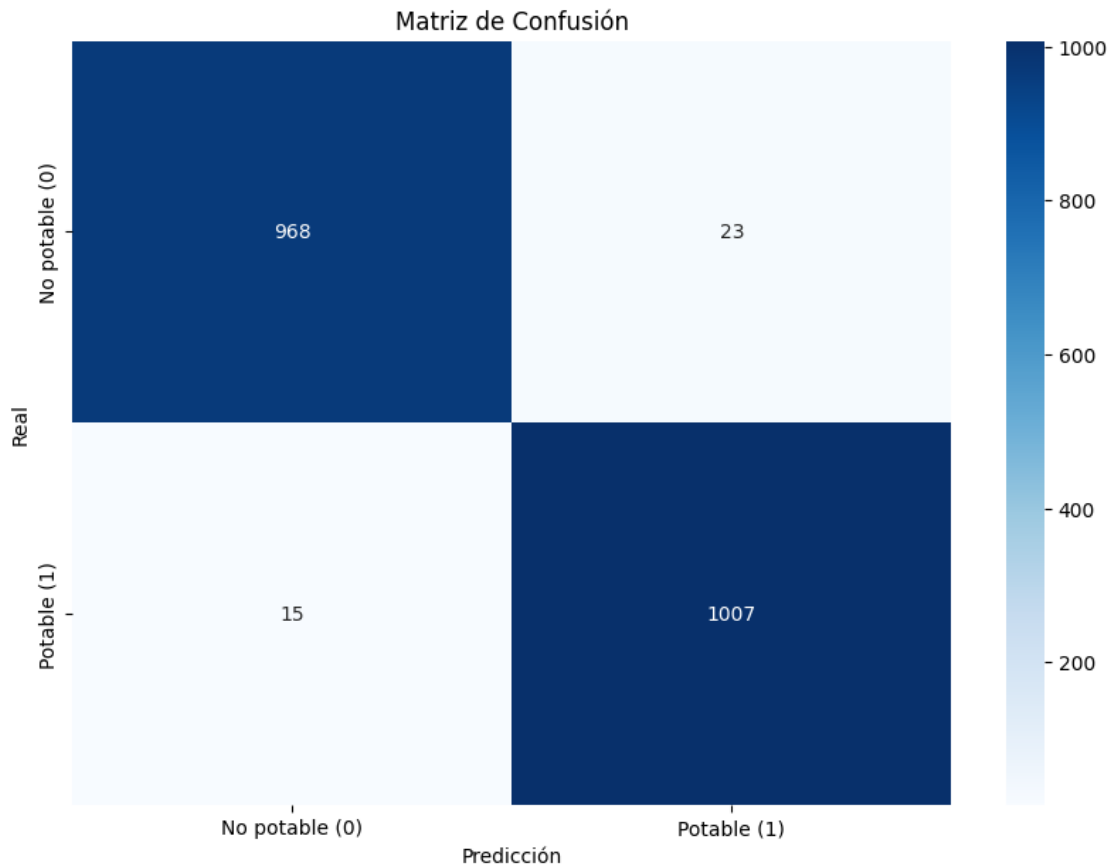
```
Best Parameters:
classifier__C: 0.1
classifier__penalty: l2
classifier__solver: liblinear
preprocessor__cat__encoder: OneHotEncoder(handle_unknown='ignore')
```

A continuación observamos las métricas de evaluación del mejor modelo.

[145]:
```
# calcular la predicción en el conjunto de prueba
y_pred = best_model_logreg.predict(X_test)

# calcular las métricas y mostrar
metrics_logreg = {"accuracy":accuracy_score(y_test, y_pred),
           "recall": recall_score(y_test, y_pred),
           "precision": precision_score(y_test, y_pred),
           "f1_score": f1_score(y_test, y_pred)}

print("Metrics:\n", metrics_logreg)

# calcular el reporte de clasificación
report_logreg = classification_report(y_test, y_pred)
print("Classification Report:\n", report_logreg)
report_logreg_df = classification_report(y_test, y_pred, output_dict=True)
report_logreg_df = pd.DataFrame(report_logreg_df).transpose()

# obtener la matriz de confusión
```

```
labels2text = {0: 'No potable (0)', 1: 'Potable (1)'}
conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 7))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',␣
↪xticklabels=[labels2text[i] for i in best_model_logreg.classes_],␣
↪yticklabels=[labels2text[i] for i in best_model_logreg.classes_])
plt.title('Matriz de Confusión')
plt.xlabel('Predicción')
plt.ylabel('Real')
plt.show()
```

```
Metrics:
 {'accuracy': 0.9811227024341779, 'recall': 0.9853228962818004, 'precision':
0.9776699029126213, 'f1_score': 0.9814814814814815}
Classification Report:
               precision    recall  f1-score   support

           0       0.98      0.98      0.98       991
           1       0.98      0.99      0.98      1022

    accuracy                           0.98      2013
   macro avg       0.98      0.98      0.98      2013
weighted avg       0.98      0.98      0.98      2013
```



1.4 4. Desarrollar un modelo basado en máquinas de vectores de soporte, incorporando una búsqueda de hiperparámetros, explorando el parámetro de regularización y el kernel. Este proceso debe estructurarse mediante un pipeline, integrando el preprocesamiento y el modelo en un flujo de trabajo reproducible y ordenado.

Definimos la grilla de hiperparámetros a explorar para el modelo de SVM, que incluye: - kernel: linear, rbf, poly y sigmoid. - C (penalización): 0.001, 0.01, 0.1, 1, 10 y 100. - Codificación de la variable categórica "Seagrass_or_algae_cover": One-Hot Encoding y Ordinal Encoding.

[146]:
```
# creamos el pipeline de SVM
pipeline_svm = Pipeline([
    ('preprocessor', preprocessor_svm),
    ('svc', SVC(kernel='poly', C=1.0))
])

# definimos la grilla de hiperparámetros
param_grid_svm = {
    'svc__kernel': ['linear', 'poly', 'rbf', 'sigmoid'],
```

```
    'svc__C': [0.1, 1, 10, 100],
    'preprocessor__cat__encoder': [
        OneHotEncoder(handle_unknown='ignore'),
        OrdinalEncoder(categories=[['Low', 'Medium', 'High']]) # los tipos de␣
 ↪categoría se ordenan para que tenga en cuenta la relación de orden
    ]
}

# hacer la búsqueda con gridseach
grid_search = GridSearchCV(
    estimator=pipeline_svm,
    param_grid=param_grid_svm,
    cv=5,
    scoring='accuracy', # usar accuracy como métrica de evaluación y 5 folds
    n_jobs=-1, # usar todos los núcleos disponibles para acelerar la búsqueda
)

grid_search.fit(X_train, y_train)
```

[146]:
```
GridSearchCV(cv=5,
             estimator=Pipeline(steps=[('preprocessor',
                                        ColumnTransformer(transformers=[('num',
Pipeline(steps=[('imputer',
          SimpleImputer()),
         ('scaler',
          StandardScaler())]),
Index(['Temperature', 'pH', 'Salinity', 'Turbidity', 'Dissolved_Oxygen',
       'Nitrate', 'Chlorophyll_a', 'pH_deviation', 'Rainfall_last_7_days_mm',
       'Distance_to_pollution_source_km', 'Water_d…
          OneHotEncoder(handle_unknown='ignore'))]),
Index(['Seagrass_or_algae_cover'], dtype='object'))])),
                                       ('svc', SVC(kernel='poly'))]),
             n_jobs=-1,
             param_grid={'preprocessor__cat__encoder':
[OneHotEncoder(handle_unknown='ignore'),
OrdinalEncoder(categories=[['Low',
    'Medium',
    'High']])],
                         'svc__C': [0.1, 1, 10, 100],
                         'svc__kernel': ['linear', 'poly', 'rbf', 'sigmoid']},
             scoring='accuracy')
```

Obtenemos el mejor modelo y los mejores hiperparámetros a través de la búsqueda con validación cruzada.

[147]:
```
# obtener el mejor modelo y parámetros
best_params_svm = grid_search.best_params_
best_model_svm = grid_search.best_estimator_
```

```
print("Best Parameters:")
for param_name, param_value in best_params_svm.items():
    print(f"{param_name}: {param_value}")
```

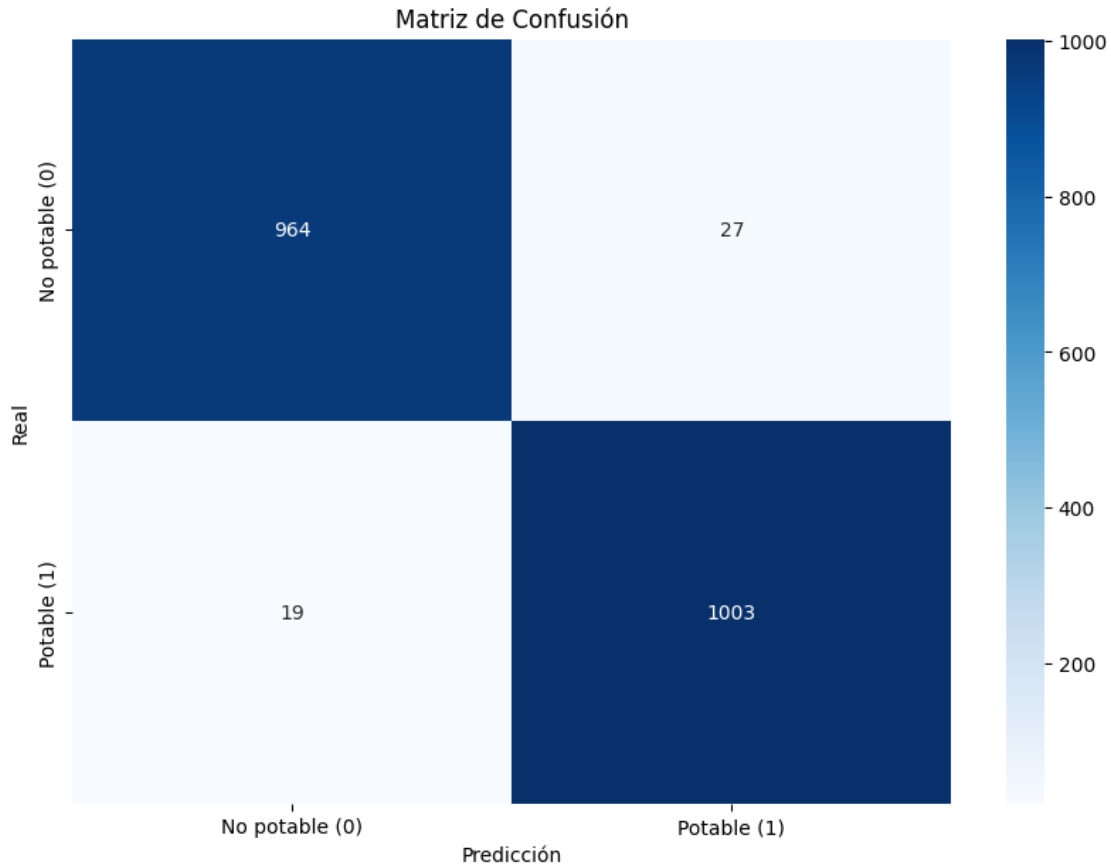
```
Best Parameters:
preprocessor__cat__encoder: OneHotEncoder(handle_unknown='ignore')
svc__C: 1
svc__kernel: poly
```

Calculamos las métricas de evaluación del mejor modelo de SVM.

[148]:
```
# calcular la predicción en el conjunto de prueba
y_pred = best_model_svm.predict(X_test)

# calcular las métricas y mostrar
metrics_svm = {"accuracy":accuracy_score(y_test, y_pred),
           "recall": recall_score(y_test, y_pred),
           "precision": precision_score(y_test, y_pred),
           "f1_score": f1_score(y_test, y_pred)}

print("Metrics:\n", metrics_svm)

# calcular el reporte de clasificación
report_svm = classification_report(y_test, y_pred)
print("Classification Report:\n", report_svm)
report_svm_df = classification_report(y_test, y_pred, output_dict=True)
report_svm_df = pd.DataFrame(report_svm_df).transpose()

# obtener la matriz de confusión
labels2text = {0: 'No potable (0)', 1: 'Potable (1)'}
conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 7))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',␣
 ↪xticklabels=[labels2text[i] for i in best_model_svm.classes_],␣
 ↪yticklabels=[labels2text[i] for i in best_model_svm.classes_])
plt.title('Matriz de Confusión')
plt.xlabel('Predicción')
plt.ylabel('Real')
plt.show()
```

```
Metrics:
 {'accuracy': 0.9771485345255837, 'recall': 0.9814090019569471, 'precision':
0.9737864077669903, 'f1_score': 0.9775828460038987}
Classification Report:
               precision    recall  f1-score   support

           0       0.98      0.97      0.98       991
           1       0.97      0.98      0.98      1022
```

```
    accuracy                           0.98      2013
   macro avg       0.98      0.98      0.98      2013
weighted avg       0.98      0.98      0.98      2013
```



1.5 5. Desarrollar un modelo basado árboles de decisión, incorporando una búsqueda de hiperparámetros con validación cruzada que permita explorar aspectos como la profundidad máxima del árbol, el número mínimo de muestras por nodo y los criterios de división. Este proceso debe estructurarse mediante un pipeline, integrando el preprocesamiento y el modelo en un flujo de trabajo reproducible y ordenado.

Definimos la grilla de hiperparámetros a explorar para el modelo de SVM, que incluye: - max_depth: 3, 5, 10, 15, 30, 60, 100 y None. - min_samples_split: 2, 5, 10, 20, 50, 100 y None. - criterion: gini, entropy y log_loss.

[149]:
```
# creamos el pipeline de DecisionTree
pipeline_dt = Pipeline([
    ('preprocessor', preprocessor_tree),
```

```python
    ('dt', DecisionTreeClassifier())
])

# definimos la grilla de hiperparámetros
param_grid_dt = {
    'dt__max_depth': [3, 5, 10, 15, 30, 60, 100, None],
    'dt__min_samples_split': [2, 5, 10, 20, 50, 100, None],
    'dt__criterion': ['gini', 'entropy', 'log_loss'],
}

# hacer la búsqueda con gridseach
grid_search = GridSearchCV(
    estimator=pipeline_dt,
    param_grid=param_grid_dt,
    cv=5,
    scoring='accuracy', # usar accuracy como métrica de evaluación y 5 folds
    n_jobs=-1, # usar todos los núcleos disponibles para acelerar la búsqueda
)

grid_search.fit(X_train, y_train)
```

[149]:
```
GridSearchCV(cv=5,
             estimator=Pipeline(steps=[('preprocessor',
                                        ColumnTransformer(transformers=[('num',
Pipeline(steps=[('imputer',
          SimpleImputer())]),
Index(['Temperature', 'pH', 'Salinity', 'Turbidity', 'Dissolved_Oxygen',
       'Nitrate', 'Chlorophyll_a', 'pH_deviation', 'Rainfall_last_7_days_mm',
       'Distance_to_pollution_source_km', 'Water_depth_m',
       'Current_velocity_m_…
Pipeline(steps=[('encoder',
          OrdinalEncoder(categories=[['Low',
                                      'Medium',
                                      'High']]))]),
Index(['Seagrass_or_algae_cover'], dtype='object'))])),
                                        ('dt', DecisionTreeClassifier())]),
             n_jobs=-1,
             param_grid={'dt__criterion': ['gini', 'entropy', 'log_loss'],
                         'dt__max_depth': [3, 5, 10, 15, 30, 60, 100, None],
                         'dt__min_samples_split': [2, 5, 10, 20, 50, 100,
                                                   None]},
             scoring='accuracy')
```

Obtenemos el mejor modelo y los mejores hiperparámetros a través de la búsqueda con validación cruzada.

[150]:
```
# obtener el mejor modelo y parámetros
best_params_dt = grid_search.best_params_
best_model_dt = grid_search.best_estimator_

print("Best Parameters:")
for param_name, param_value in best_params_dt.items():
    print(f"{param_name}: {param_value}")
```

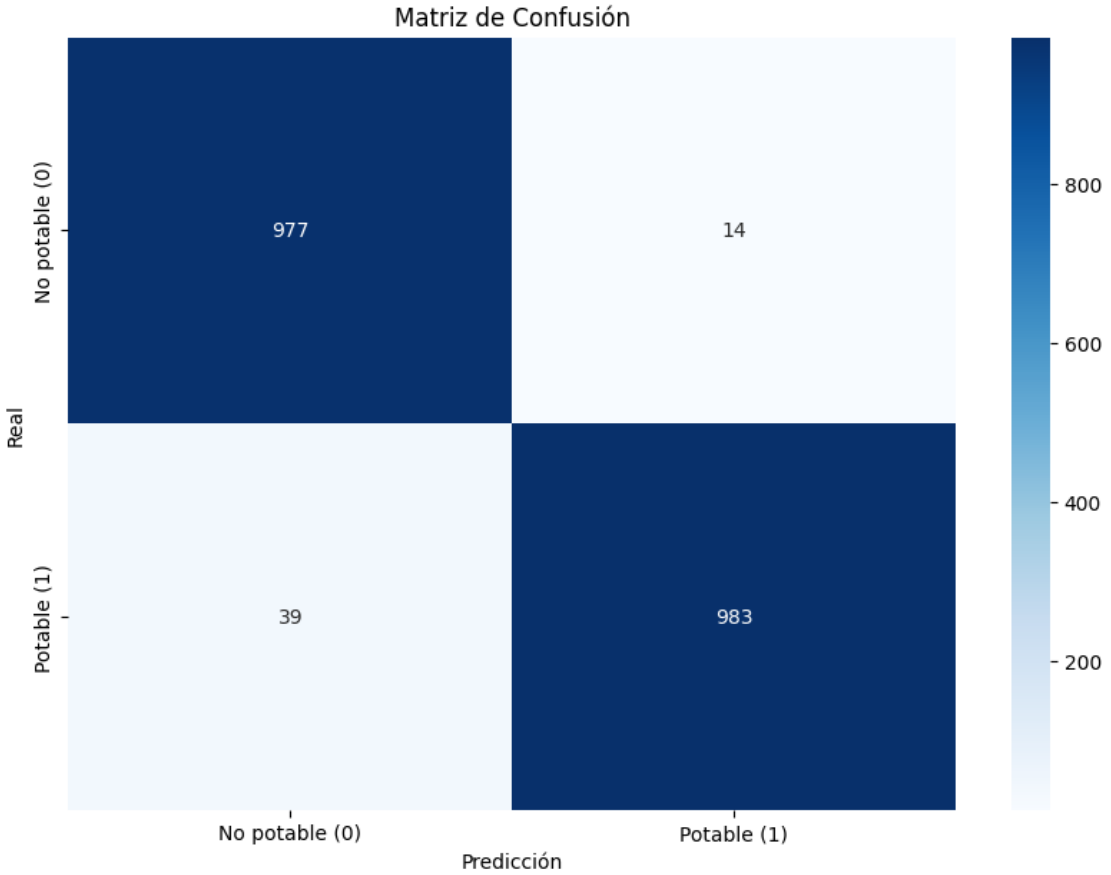
```
Best Parameters:
dt__criterion: gini
dt__max_depth: 5
dt__min_samples_split: 100
```

Obtenemos las métricas de evaluación del mejor modelo de árboles de decisión.

[151]:
```
# calcular la predicción en el conjunto de prueba
y_pred = best_model_dt.predict(X_test)

# calcular las métricas y mostrar
metrics_dt = {"accuracy":accuracy_score(y_test, y_pred),
           "recall": recall_score(y_test, y_pred),
           "precision": precision_score(y_test, y_pred),
           "f1_score": f1_score(y_test, y_pred)}

print("Metrics:\n", metrics_dt)

# calcular el reporte de clasificación
report_dt = classification_report(y_test, y_pred)
print("Classification Report:\n", report_dt)
report_dt_df = classification_report(y_test, y_pred, output_dict=True)
report_dt_df = pd.DataFrame(report_dt_df).transpose()

# obtener la matriz de confusión
labels2text = {0: 'No potable (0)', 1: 'Potable (1)'}
conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 7))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',␣
 ↪xticklabels=[labels2text[i] for i in best_model_dt.classes_],␣
 ↪yticklabels=[labels2text[i] for i in best_model_dt.classes_])
plt.title('Matriz de Confusión')
plt.xlabel('Predicción')
plt.ylabel('Real')
plt.show()
```

```
Metrics:
 {'accuracy': 0.9736711376055638, 'recall': 0.961839530332681, 'precision':
0.9859578736208626, 'f1_score': 0.9737493808816245}
Classification Report:
               precision    recall  f1-score   support
```

```
           0       0.96      0.99      0.97       991
           1       0.99      0.96      0.97      1022

    accuracy                           0.97      2013
   macro avg       0.97      0.97      0.97      2013
weighted avg       0.97      0.97      0.97      2013
```



1.6 6. Elaboración de una tabla comparativa mostrando el rendimiento sobre test de los mejores dos modelos obtenidos en los puntos 3, 4 y 5, con base en las métricas recall, precisión y F1-score.

A continuación se presenta un dataframe comparativo con las métricas de evaluación (recall, precisión y F1-score) para los mejores modelos de regresión logística, SVM y árboles de decisión.

[152]:
```
# crear un df para la tabla comparativa para cada clase (0, 1) y cada métrica
comparison_df = pd.DataFrame({
    'Model': ['Logistic Regression', 'SVM', 'Decision Tree'],
```

```
    'Recall_1':    [report_logreg_df.loc['1', 'recall'],     report_svm_df.
 ↪loc['1', 'recall'],     report_dt_df.loc['1', 'recall']],
    'Precision_1': [report_logreg_df.loc['1', 'precision'],  report_svm_df.
 ↪loc['1', 'precision'],  report_dt_df.loc['1', 'precision']],
    'F1_1':        [report_logreg_df.loc['1', 'f1-score'],   report_svm_df.
 ↪loc['1', 'f1-score'],   report_dt_df.loc['1', 'f1-score']],
    'Recall_0':    [report_logreg_df.loc['0', 'recall'],     report_svm_df.
 ↪loc['0', 'recall'],     report_dt_df.loc['0', 'recall']],
    'Precision_0': [report_logreg_df.loc['0', 'precision'],  report_svm_df.
 ↪loc['0', 'precision'],  report_dt_df.loc['0', 'precision']],
    'F1_0':        [report_logreg_df.loc['0', 'f1-score'],   report_svm_df.
 ↪loc['0', 'f1-score'],   report_dt_df.loc['0', 'f1-score']],
})
comparison_df
```

[152]:

```
                 Model  Recall_1  Precision_1      F1_1  Recall_0  \
0  Logistic Regression  0.985323     0.977670  0.981481  0.976791
1                  SVM  0.981409     0.973786  0.977583  0.972755
2        Decision Tree  0.961840     0.985958  0.973749  0.985873

   Precision_0      F1_0
0     0.984741  0.980750
1     0.980671  0.976697
2     0.961614  0.973592
```

1.7 7. Identificar las variables más relevantes para la predicción utilizando tanto el modelo de árboles de decisión como el de regresión logística. Posteriormente, comparar la importancia de las variables según cada modelo, con el fin de analizar cómo distintos enfoques destacan diferentes factores asociados a la calidad del agua y así comprender mejor qué características influyen en que un cuerpo de agua sea considerado seguro o no.

Para el modelo de árboles de decisión, se obtiene la importancia de las variables utilizando el atributo "feature_importances_" del modelo y visualizando el árbol, el cual indica que entre más alta es la importancia de una variable, más cerca del nodo raíz se encuentra.

[153]:

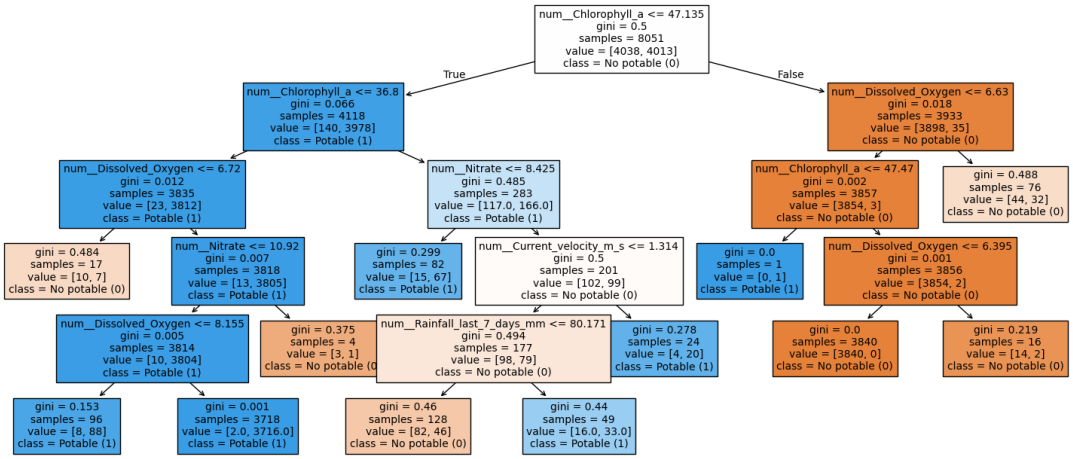
```
# obtener importancia de variables en DecisionTree
feature_importances_dt = best_model_dt.named_steps['dt'].feature_importances_
feature_names_dt = best_model_dt.named_steps['preprocessor'].
 ↪get_feature_names_out()
importance_df_dt = pd.DataFrame({'Feature': feature_names_dt, 'Importance':␣
 ↪feature_importances_dt}).sort_values(by='Importance', ascending=False)

importance_df_dt
```

[153]:

```
                                Feature  Importance
6                    num__Chlorophyll_a    0.981861
```

```
4                 num__Dissolved_Oxygen    0.010322
5                          num__Nitrate    0.004352
8           num__Rainfall_last_7_days_mm    0.001818
11             num__Current_velocity_m_s    0.001647
0                      num__Temperature    0.000000
1                                num__pH    0.000000
2                          num__Salinity    0.000000
3                         num__Turbidity    0.000000
7                      num__pH_deviation    0.000000
9   num__Distance_to_pollution_source_km    0.000000
10                    num__Water_depth_m    0.000000
12     num__Phosphate_concentration_mg_L    0.000000
13                         num__BOD_mg_L    0.000000
14             num__Distance_to_coast_km    0.000000
15          cat__Seagrass_or_algae_cover    0.000000
```

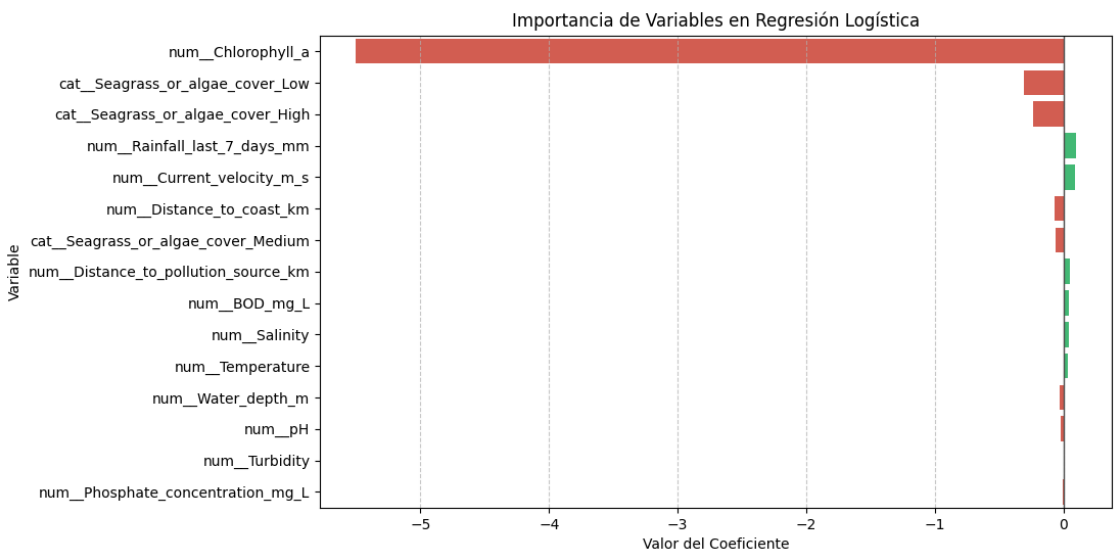
[154]:
```
# visualizar el árbol de decisión
plt.figure(figsize=(19,8))
plot_tree(best_model_dt.named_steps['dt'], feature_names=feature_names_dt,␣
 ↪class_names=['No potable (0)', 'Potable (1)'], filled=True, fontsize=10)
plt.show()
```



En base a los resultados, las variables más importantes para el modelo de árboles de decisión son `Clorophyll_a`, `Dissolved_Oxygen` y `Nitrate`.

Para identificar las variables más relevantes de la regresión logística, se pueden analizar los coeficientes del modelo. Los coeficientes indican la dirección y la magnitud del impacto de cada variable en la probabilidad de que el agua sea potable. Entre más alto sea el valor absoluto del coeficiente, mayor será la importancia de esa variable en la predicción. Se puede crear un DataFrame que muestre los nombres de las variables junto con sus coeficientes para facilitar la comparación.

[158]:
```
# importancia de variables en regresión logística
feature_names_logreg = best_model_logreg.named_steps['preprocessor'].
 ↪get_feature_names_out()
coefficients_logreg = best_model_logreg.named_steps['classifier'].coef_[0]
importance_df_logreg = pd.DataFrame({'Feature': feature_names_logreg,␣
 ↪'Coefficient': coefficients_logreg}).sort_values(by='Coefficient', key=abs,␣
 ↪ascending=False)

# plot de importancia de variables en regresión logística
colors = ['#2ecc71' if c > 0 else '#e74c3c' for c in␣
 ↪importance_df_logreg['Coefficient']]

plt.figure(figsize=(10,6))
plt.axvline(x=0, color='.3', linewidth=1)
sns.barplot(x='Coefficient', y='Feature', data=importance_df_logreg,␣
 ↪palette=colors)
plt.title('Importancia de Variables en Regresión Logística')
plt.xlabel('Valor del Coeficiente')
plt.ylabel('Variable')
plt.grid(True, axis='x', linestyle='--', alpha=0.7)
plt.show()
```



Para la regresión logística, las variables más importantes son `Clorophyll_a`, `Seagrass_or_algae_cover` en la categoría "Low" y "High". Es interesante observar que la variable `Clorophyll_a` es la más correlacionada a la variable objetivo tras la limpieza con el resto de correlaciones entre variables explicativas, lo que se refleja en su alta importancia para el modelo de regresión logística.

---

2 Análisis de resultados

2.1 ¿Qué puedes decir de los valores de las métricas recall y precisión para cada una de las clases en cada modelo? ¿Qué métrica considera más adecuada para este problema y por qué, teniendo en cuenta que se trata de clasificar si el agua es segura o no?

La métrica recall o también llamada true positive rate, mide cuántos ejemplos de la clase positiva son correctamente identificados por el modelo. En este contexto, el recall para la clase positiva (agua potable) indica de las muestras que son realmente agua potable, cuántas son correctamente clasificadas como potable, mientras que el recall para la clase negativa (agua no potable) indica de las muestras que son realmente agua no potable, cuántas son correctamente clasificadas como no potable. Este segundo tipo de error (muestras clasificadas incorrectamente como agua potable) puede ser más perjudicial en el contexto, pues clasificar como agua potable una muestra que no lo es tiene consecuencias graves en la salud del consumidor del agua.

Los resultados para cada modelo sobre esta métrica y sobre ambas clases son los siguientes:

[156]:
```
comparison_df[['Model','Recall_0', 'Recall_1']]
```

[156]:

	Model	Recall_0	Recall_1
0	Logistic Regression	0.976791	0.985323
1	SVM	0.972755	0.981409
2	Decision Tree	0.985873	0.961840

Con estos resultados, podemos concluir que el modelo Decision Tree es el mejor identificando el agua no potable, con un recall de aproximadamente 0.98, mientras que el modelo de regresión logística es el que mejor identifica el agua potable, con un recall de aproximadamente 0.98.

Por otro lado, la métrica de precisión mide cuántos ejemplos que el modelo predice como positivos, o en este caso, como agua potable, son realmente positivos, y para el caso negativo, cuántos ejemplos que el modelo predice como negativos, o en este caso, como agua no potable, son realmente negativos. En este contexto, una precisión alta en la clase 0 indica que cuando el modelo alerta sobre agua no segura, esa alerta es confiable y no genera "falsas alarmas" sobre el agua no potable. Una precisión baja, en cambio, significaría que el modelo es demasiado conservador, descartando muestras que en realidad eran seguras, lo que tiene un costo, pero no afecta la salud de los consumidores. En la clase 1, una precisión alta garantiza que el agua distribuida como segura efectivamente lo es.

Los resultados sobre cada modelo y cada clase son los siguientes:

[157]:
```
comparison_df[['Model','Precision_0', 'Precision_1']]
```

[157]:

	Model	Precision_0	Precision_1
0	Logistic Regression	0.984741	0.977670
1	SVM	0.980671	0.973786
2	Decision Tree	0.961614	0.985958

Para esta métrica, el modelo de regresión logística es el que mejor identifica las muestras de agua no potable, mientras que el modelo de árboles de decisión es el que mejor identifica las muestras de agua potable.

Si consideramos que el propósito del modelo es identificar correctamente las muestras de agua potable, y que es más perjudicial clasificar como agua potable una muestra que no lo es, la **métrica más adecuada para este problema sería el recall** para la clase 0, ya que esta métrica penaliza fuertemente el hecho de clasificar agua no potable como potable. Teniendo esto en cuenta, el modelo a elegir será aquel con un recall más alto para la clase 0.

2.2 Si los tres modelos proporcionan resultados similares en cuanto a las métricas de rendimiento ¿Cuál seleccionaría tomando en cuenta el contexto del problema?

Efectivamente, los tres modelos presentan un rendimiento muy similar, con diferencias mínimas entre sus métricas de evaluación. Considerando el contexto del problema, donde el principal riesgo es distribuir agua contaminada como si fuera segura, con consecuencias directas sobre la salud de la población y los ecosistemas acuáticos, el modelo seleccionado es el **árbol de decisión**. Este modelo obtiene el recall más alto para la clase 0 (agua no segura), con un valor aproximado de 0.98, lo que significa que es el más efectivo identificando muestras contaminadas y, por tanto, el que minimiza en mayor medida el riesgo de un falso positivo. En un problema donde el costo de no detectar agua no potable supera ampliamente al de generar una falsa alarma, esta característica resulta determinante para la selección del modelo.

2.3 ¿Coinciden los modelos en identificar las mismas variables como importantes? ¿Qué variables aparecen de manera consistente en ambos modelos?

Los tres modelos coinciden en señalar `Chlorophyll_a` como la variable más importante para clasificar la potabilidad del agua, lo cual es consistente con la correlación observada entre esta variable y la variable objetivo. Esto tiene sentido desde el punto de vista ecológico: niveles elevados de clorofila indican un crecimiento excesivo de algas, lo que reduce el oxígeno disuelto y eleva la concentración de nutrientes, degradando la calidad del agua. Este mismo fenómeno se refleja en la posición de `Seagrass_or_algae_cover` como segunda variable más importante en la regresión logística, reforzando la idea de que la presencia de biomasa algal es un indicador relevante de agua no segura. A partir de la segunda posición, los modelos difieren en el orden de importancia de las variables. Esto se explica en parte por las decisiones de preprocesamiento: en la regresión logística se eliminaron variables correlacionadas entre sí para mitigar el efecto de la multicolinealidad, mientras que el árbol de decisión contó con el conjunto completo de variables, lo que le permite capturar relaciones adicionales entre ellas; para el árbol de decisión, `Dissolved_Oxygen` y `Nitrate` aparecen como las siguientes variables más importantes, lo que es coherente con su alta correlación con la variable objetivo, mientras que en la regresión logística, al eliminar estas variables, se destacan otras como `Seagrass_or_algae_cover` y `Rainfall_last_7_days_mm`.

2.4 Si una autoridad ambiental quisiera mejorar la calidad del agua, ¿sobre qué variables debería poner mayor atención según el análisis realizado?

Con base en el análisis, las variables más relevantes para la potabilidad del agua son `Chlorophyll_a`, `Dissolved_Oxygen` y `Nitrate`. Estas tres variables están estrechamente relacionadas, pues el exceso de nitratos suele provenir principalmente de deshechos agrícolas y vertimientos urbanos, los cuales favorecen el crecimiento de algas, elevando los niveles de clorofila y reduciendo el oxígeno disuelto en el agua necesario para la vida acuática. Este proceso se conoce como eutrofización, y es uno de los principales causantes de degradación de cuerpos de agua. Por lo tanto, una estrategia efectiva para mejorar la calidad del agua se debería enfocar en el primer factor de la cadena, regulando

actividades agrícolas e industriales cercanas a fuentes de agua. Además, se deberían monitorear continuamente las variables ya mencionadas para detectar de forma oportuna deterioros de agua que se puedan prevenir.

[2]:
```
# convertir a pdf
!jupyter nbconvert --to pdf e.laverdeb@uniandes.edu.co_parcial1.ipynb --output␣
↪parcial1.pdf
```

```
[NbConvertApp] Converting notebook e.laverdeb@uniandes.edu.co_parcial1.ipynb to
pdf
[NbConvertApp] Support files will be in parcial1_files/
[NbConvertApp] Making directory ./parcial1_files
[NbConvertApp] Writing 1794860 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no
citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 1071046 bytes to parcial1.pdf
```